Adaptive Question Difficulty with LLM

Objective

Dynamically generate quiz questions from **lecture content** at difficulty levels adapted to each student's wellbeing and academic performance. High-risk students receive easier questions to reduce stress, while low-risk students are challenged to maintain engagement.

How It Works

1. Teacher uploads a lecture PDF
2. System extracts and processes the content
3. Student's risk label + exam score determines difficulty level
4. Gemini Flash generates questions FROM the lecture at that difficulty
5. System adapts difficulty based on student's quiz performance

Section 1: Install and Import Librares

In [28]:

```
# ============================================================
# SECTION 1: INSTALL AND IMPORT LIBRARIES
# ============================================================
!pip install google-generativeai PyPDF2 pdfplumber pandas matplotlib seaborn

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import re
import time
import warnings
warnings.filterwarnings('ignore')

try:
    import pdfplumber
    HAS_PDF = True
    print("[OK] pdfplumber loaded")
except:
    HAS_PDF = False
    print("[WARN] pdfplumber not found. Install: pip install pdfplumber")

try:
    import google.generativeai as genai
    HAS_GENAI = True
    print("[OK] google-generativeai loaded")
except:
    HAS_GENAI = False
    print("[WARN] google-generativeai not found. Install: pip install google-g
```

```
plt.style.use('seaborn-v0_8-whitegrid')

print("\n" + "=" * 60)
print("COLAB 2: ADAPTIVE QUESTION DIFFICULTY WITH LLM")
print("=" * 60)
```

```
Requirement already satisfied: google-generativeai in /usr/local/lib/python3.1
2/dist-packages (0.8.6)
Requirement already satisfied: PyPDF2 in /usr/local/lib/python3.12/dist-package
s (3.0.1)
Requirement already satisfied: pdfplumber in /usr/local/lib/python3.12/dist-pac
kages (0.11.9)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-package
s (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-pac
kages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packag
es (0.13.2)
Requirement already satisfied: google-ai-generativelanguage==0.6.15 in /usr/loc
al/lib/python3.12/dist-packages (from google-generativeai) (0.6.15)
Requirement already satisfied: google-api-core in /usr/local/lib/python3.12/dis
t-packages (from google-generativeai) (2.30.3)
Requirement already satisfied: google-api-python-client in /usr/local/lib/pytho
n3.12/dist-packages (from google-generativeai) (2.194.0)
Requirement already satisfied: google-auth>=2.15.0 in /usr/local/lib/python3.1
2/dist-packages (from google-generativeai) (2.47.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packa
ges (from google-generativeai) (5.29.6)
Requirement already satisfied: pydantic in /usr/local/lib/python3.12/dist-packa
ges (from google-generativeai) (2.12.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
(from google-generativeai) (4.67.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/d
ist-packages (from google-generativeai) (4.15.0)
Requirement already satisfied: proto-plus<2.0.0dev,>=1.22.3 in /usr/local/lib/p
ython3.12/dist-packages (from google-ai-generativelanguage==0.6.15->google-gene
rativeai) (1.27.2)
Requirement already satisfied: pdfminer.six==20251230 in /usr/local/lib/python
3.12/dist-packages (from pdfplumber) (20251230)
Requirement already satisfied: Pillow>=9.1 in /usr/local/lib/python3.12/dist-pa
ckages (from pdfplumber) (11.3.0)
Requirement already satisfied: pypdfium2>=4.18.0 in /usr/local/lib/python3.12/d
ist-packages (from pdfplumber) (5.8.0)
Requirement already satisfied: charset-normalizer>=2.0.0 in /usr/local/lib/pyth
on3.12/dist-packages (from pdfminer.six==20251230->pdfplumber) (3.4.7)
Requirement already satisfied: cryptography>=36.0.0 in /usr/local/lib/python3.1
2/dist-packages (from pdfminer.six==20251230->pdfplumber) (43.0.3)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-
packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python
3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-p
ackages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis
t-packages (from pandas) (2026.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-p
ackages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/d
```

```
ist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/d
ist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dis
t-packages (from matplotlib) (26.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/di
st-packages (from matplotlib) (3.3.2)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.63.2 in /usr/
local/lib/python3.12/dist-packages (from google-api-core->google-generativeai)
(1.74.0)
Requirement already satisfied: requests<3.0.0,>=2.20.0 in /usr/local/lib/python
3.12/dist-packages (from google-api-core->google-generativeai) (2.32.4)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python
3.12/dist-packages (from google-auth>=2.15.0->google-generativeai) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-
packages (from google-auth>=2.15.0->google-generativeai) (4.9.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packa
ges (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in /usr/local/lib/python
3.12/dist-packages (from google-api-python-client->google-generativeai) (0.3
1.2)
Requirement already satisfied: google-auth-httplib2<1.0.0,>=0.2.0 in /usr/loca
l/lib/python3.12/dist-packages (from google-api-python-client->google-generativ
eai) (0.3.1)
Requirement already satisfied: uritemplate<5,>=3.0.1 in /usr/local/lib/python
3.12/dist-packages (from google-api-python-client->google-generativeai) (4.2.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python
3.12/dist-packages (from pydantic->google-generativeai) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python
3.12/dist-packages (from pydantic->google-generativeai) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/pytho
n3.12/dist-packages (from pydantic->google-generativeai) (0.4.2)
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.12/dist-pac
kages (from cryptography>=36.0.0->pdfminer.six==20251230->pdfplumber) (2.0.0)
Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in /usr/local/lib/python
3.12/dist-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.1
0.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,!=2.8.*,!=2.9.*,<3.0.0de
v,>=1.34.1->google-ai-generativelanguage==0.6.15->google-generativeai) (1.80.0)
Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in /usr/local/lib/p
ython3.12/dist-packages (from google-api-core[grpc]!=2.0.*,!=2.1.*,!=2.1
0.*,!=2.2.*,!=2.3.*,!=2.4.*,!=2.5.*,!=2.6.*,!=2.7.*,!=2.8.*,!=2.9.*,<3.0.0de
v,>=1.34.1->google-ai-generativelanguage==0.6.15->google-generativeai) (1.71.2)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.1
2/dist-packages (from pyasn1-modules>=0.2.1->google-auth>=2.15.0->google-genera
tiveai) (0.6.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-p
ackages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai)
(3.13)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/
dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generative
ai) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/
dist-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generative
ai) (2026.4.22)
```

```
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-pack
ages (from cffi>=1.12->cryptography>=36.0.0->pdfminer.six==20251230->pdfplumbe
r) (3.0)
[OK] pdfplumber loaded
[OK] google-generativeai loaded

============================================================
COLAB 2: ADAPTIVE QUESTION DIFFICULTY WITH LLM
============================================================
```

Section 2: Configure GEMINI API

In [29]:
```python
# ============================================================
# SECTION 2: CONFIGURE GEMINI API
# ============================================================
# Replace with your actual API key
GEMINI_API_KEY = "AIzaSyBcS7uKdip-I3koN6B6VqU8LrEILsEXFnU"

if HAS_GENAI and GEMINI_API_KEY != "YOUR_KEY_HERE":
    genai.configure(api_key=GEMINI_API_KEY)
    model = genai.GenerativeModel('gemini-2.5-flash')
    print("[OK] Gemini Flash API configured successfully")
else:
    model = None
    print("[INFO] Gemini API not configured. Set GEMINI_API_KEY above.")
    print("       Demo mode will use sample questions instead.")
```

```
[OK] Gemini Flash API configured successfully
```

Section 3: PDF Lecture Content Extraction

In [30]:
```python
# ============================================================
# SECTION 3: PDF LECTURE CONTENT EXTRACTION
# ============================================================
from google.colab import files; uploaded = files.upload()
# Then set LECTURE_PDF_PATH to the uploaded file name

LECTURE_PDF_PATH = "lecture_slides.pdf"  # Update with your PDF filename

def extract_pdf_content(pdf_path):
    """Extract text content from a PDF file using pdfplumber"""
    if not HAS_PDF:
        print("[WARN] pdfplumber not available. Using sample content.")
        return None, None

    try:
        pages_content = []
        with pdfplumber.open(pdf_path) as pdf:
            for i, page in enumerate(pdf.pages):
                text = page.extract_text()
                if text:
                    # Clean the text
                    text = re.sub(r'\s+', ' ', text).strip()
```

```
                    text = re.sub(r'Page \d+', '', text)  # Remove page number
                    pages_content.append({
                        'page': i + 1,
                        'content': text,
                        'word_count': len(text.split())
                    })

        full_content = ' '.join([p['content'] for p in pages_content])
        print(f"[OK] Extracted {len(pages_content)} pages")
        print(f"     Total words: {sum(p['word_count'] for p in pages_content)
        return pages_content, full_content
    except FileNotFoundError:
        print(f"[WARN] PDF file '{pdf_path}' not found. Using sample content."
        return None, None

pages, full_content = extract_pdf_content(LECTURE_PDF_PATH)

# --- FALLBACK: Sample lecture content for demo ---
# This is used when no PDF is uploaded, so the notebook still runs end-to-end

SAMPLE_LECTURE_CONTENT = """
Introduction to Photosynthesis - Grade 8 Science

Photosynthesis is the process by which green plants and some other organisms u
sunlight to synthesize foods with carbon dioxide and water. Photosynthesis in
generally involves the green pigment chlorophyll and generates oxygen as a byp

The Chemical Equation:
6CO2 + 6H2O + light energy -> C6H12O6 + 6O2

Key Concepts:
1. Light-dependent reactions occur in the thylakoid membranes
2. Light-independent reactions (Calvin Cycle) occur in the stroma
3. Chlorophyll absorbs red and blue light, reflects green light
4. Factors affecting photosynthesis: light intensity, CO2 concentration, tempe
5. Plants use glucose for energy, growth, and storage as starch

Importance of Photosynthesis:
- Produces oxygen for all living organisms
- Forms the base of most food chains
- Removes carbon dioxide from the atmosphere
- Provides raw materials for plant growth

Experiment: Testing for Starch in Leaves
Materials: leaf, iodine solution, ethanol, boiling water
Method: Boil leaf in water, then in ethanol to remove chlorophyll.
Add iodine solution - blue-black color indicates presence of starch.

Summary: Photosynthesis is essential for life on Earth. It converts light ener
into chemical energy stored in glucose, releasing oxygen as a byproduct.
"""

if full_content is None:
```

```
    full_content = SAMPLE_LECTURE_CONTENT
    pages = [{'page': 1, 'content': SAMPLE_LECTURE_CONTENT, 'word_count': len(
    print("\n[DEMO] Using sample lecture content: 'Introduction to Photosynthe

# --- Visualize PDF content statistics ---
pages_df = pd.DataFrame(pages)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].bar(pages_df['page'], pages_df['word_count'], color='#3498db', edgecol
axes[0].set_title('Word Count per Slide/Page', fontweight='bold')
axes[0].set_xlabel('Page Number')
axes[0].set_ylabel('Word Count')

# Topic extraction (keyword frequency)
words = full_content.lower().split()
stop_words = {'the', 'is', 'in', 'and', 'of', 'to', 'a', 'for', 'that', 'are',
keywords = [w for w in words if len(w) > 3 and w not in stop_words and w.isalp
from collections import Counter
top_keywords = Counter(keywords).most_common(15)

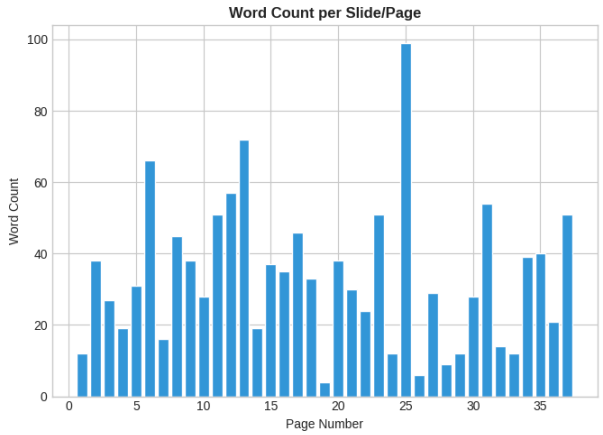
kw_names, kw_counts = zip(*top_keywords) if top_keywords else ([], [])
axes[1].barh(kw_names, kw_counts, color='#9b59b6', edgecolor='white')
axes[1].set_title('Top Keywords in Lecture Content', fontweight='bold')
axes[1].set_xlabel('Frequency')
axes[1].invert_yaxis()

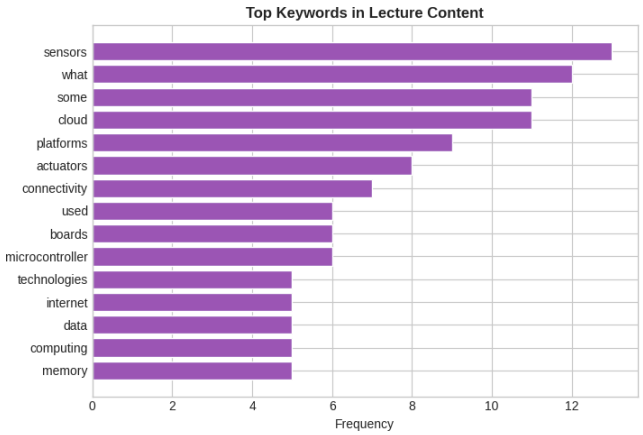
plt.tight_layout()
plt.savefig('c2_chart1_pdf_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```

Browse...

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
[OK] Extracted 37 pages
     Total words: 1243
```





Section 4: Student Difficulty Mapping

In [31]:

```
# ============================================================
# SECTION 4: LOAD STUDENT DATA & MAP DIFFICULTY LEVELS
```

```python
# ============================================================
df = pd.read_csv('student_wellbeing_dataset.csv')

# Use only the LATEST week (week 8) data for each student
# This represents their current state
latest = df[df['week'] == df.groupby('student_id')['week'].transform('max')]
latest = latest.copy()

print(f"Students: {latest.shape[0]}")
print(f"\nRisk distribution:")
print(latest['risk_label'].value_counts())

# %%
# --- Performance Level Binning ---
# Unique to this notebook: bin continuous scores into categorical levels
def assign_performance_level(score):
    if score < 50:
        return 'Low'
    elif score <= 75:
        return 'Mid'
    else:
        return 'High'

latest['performance_level'] = latest['avg_exam_score'].apply(assign_performanc

# --- Difficulty Mapping ---
difficulty_map = {
    ('HIGH', 'Low'): 'VERY_EASY',
    ('HIGH', 'Mid'): 'EASY',
    ('HIGH', 'High'): 'MEDIUM',
    ('MEDIUM', 'Low'): 'EASY',
    ('MEDIUM', 'Mid'): 'MEDIUM',
    ('MEDIUM', 'High'): 'MEDIUM_HARD',
    ('LOW', 'Low'): 'MEDIUM',
    ('LOW', 'Mid'): 'MEDIUM_HARD',
    ('LOW', 'High'): 'HARD',
}

# Bloom's taxonomy mapping
blooms_map = {
    'VERY_EASY': 'Remember',
    'EASY': 'Understand',
    'MEDIUM': 'Apply',
    'MEDIUM_HARD': 'Analyze',
    'HARD': 'Evaluate/Create'
}

latest['difficulty_level'] = latest.apply(
    lambda r: difficulty_map.get((r['risk_label'], r['performance_level']), 'M
)
latest['bloom_level'] = latest['difficulty_level'].map(blooms_map)

print("\n--- Difficulty Assignment Summary ---")
```

```
print(latest['difficulty_level'].value_counts())
print(f"\nSample assignments:")
print(latest[['student_id', 'grade', 'risk_label', 'avg_exam_score', 'performa
               'difficulty_level', 'bloom_level']].head(10).to_string(index=Fa

# %%
# --- Visualization: Difficulty Distribution ---
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Chart 1: Difficulty distribution
diff_counts = latest['difficulty_level'].value_counts()
diff_order = ['VERY_EASY', 'EASY', 'MEDIUM', 'MEDIUM_HARD', 'HARD']
diff_colors = ['#2ecc71', '#27ae60', '#f39c12', '#e67e22', '#e74c3c']
counts_ordered = [diff_counts.get(d, 0) for d in diff_order]

axes[0].bar(diff_order, counts_ordered, color=diff_colors, edgecolor='white')
axes[0].set_title('Difficulty Level Distribution', fontweight='bold')
axes[0].set_ylabel('Number of Students')
axes[0].tick_params(axis='x', rotation=45)

# Chart 2: Risk x Performance Heatmap
ct = pd.crosstab(latest['risk_label'], latest['performance_level'])
ct = ct.reindex(index=['HIGH', 'MEDIUM', 'LOW'], columns=['Low', 'Mid', 'High'
sns.heatmap(ct, annot=True, fmt='d', cmap='YlOrRd', ax=axes[1])
axes[1].set_title('Risk Level x Performance Level', fontweight='bold')

# Chart 3: Students per grade
grade_counts = latest.groupby(['grade', 'difficulty_level']).size().unstack(fi
grade_counts.plot(kind='bar', stacked=True, ax=axes[2], color=diff_colors[:len
axes[2].set_title('Difficulty by Grade', fontweight='bold')
axes[2].set_ylabel('Count')
axes[2].tick_params(axis='x', rotation=0)
axes[2].legend(fontsize=8)

plt.tight_layout()
plt.savefig('c2_chart2_difficulty_mapping.png', dpi=150, bbox_inches='tight')
plt.show()

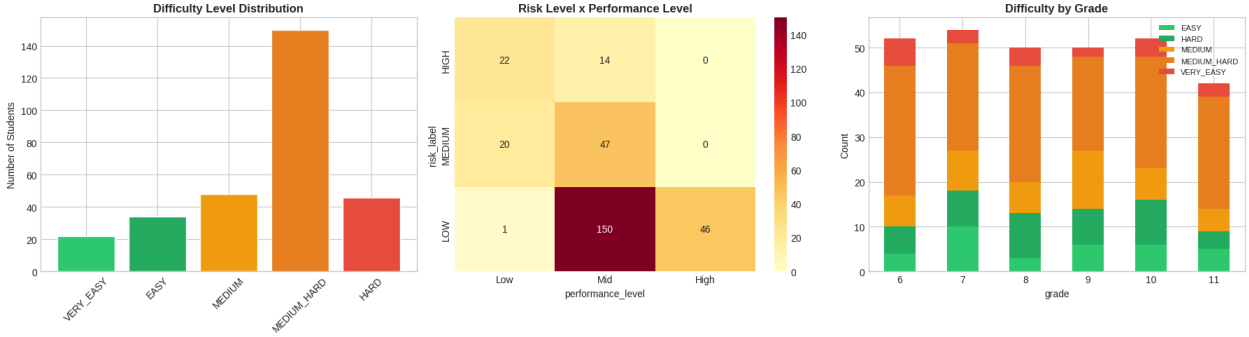
print("\n" + "=" * 60)
print("PART 1 COMPLETE: PDF extracted & difficulty mapped!")
print("Proceed to Part 2 for Quiz Generation & Analysis.")
print("=" * 60)
```

```
Students: 300

Risk distribution:
risk_label
LOW       197
MEDIUM     67
HIGH       36
Name: count, dtype: int64

--- Difficulty Assignment Summary ---
difficulty_level
MEDIUM_HARD    150
MEDIUM          48
HARD            46
EASY            34
VERY_EASY       22
Name: count, dtype: int64

Sample assignments:
student_id  grade risk_label  avg_exam_score performance_level difficulty_level
bloom_level
      S001     10       HIGH              61               Mid             EASY
Understand
      S002     10     MEDIUM              49               Low             EASY
Understand
      S003      7        LOW              68               Mid      MEDIUM_HARD
Analyze
      S004      8        LOW              67               Mid      MEDIUM_HARD
Analyze
      S005      7        LOW              95              High             HARD
Evaluate/Create
      S006      9        LOW              80              High             HARD
Evaluate/Create
      S007     10        LOW              57               Mid      MEDIUM_HARD
Analyze
      S008      8        LOW              76              High             HARD
Evaluate/Create
      S009      6        LOW              69               Mid      MEDIUM_HARD
Analyze
      S010      9     MEDIUM              45               Low             EASY
Understand
```



```
============================================================
PART 1 COMPLETE: PDF extracted & difficulty mapped!
Proceed to Part 2 for Quiz Generation & Analysis.
============================================================
```

Section 5: Gemini Flash Quiz Generation Pipeline

In [32]:

```python
# ============================================================
# SETUP (Run after Part 1 or standalone)
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import json
import re
import time
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

try:
    import google.generativeai as genai
    HAS_GENAI = True
except: HAS_GENAI = False

plt.style.use('seaborn-v0_8-whitegrid')

# --- CONFIGURE API (set your key) ---
GEMINI_API_KEY = "AIzaSyBcS7uKdip-I3koN6B6VqU8LrEILsEXFnU"

if HAS_GENAI and GEMINI_API_KEY != "YOUR_KEY_HERE":
    genai.configure(api_key=GEMINI_API_KEY)
    gemini_model = genai.GenerativeModel('gemini-2.5-flash')
    API_AVAILABLE = True
else:
    gemini_model = None
    API_AVAILABLE = False
    print("[INFO] Running in DEMO mode (no API key). Using sample questions.")

# --- Load data & recreate mappings ---
df = pd.read_csv('student_wellbeing_dataset.csv')
latest = df[df['week'] == df.groupby('student_id')['week'].transform('max')].c

def assign_perf(s):
    return 'Low' if s < 50 else ('Mid' if s <= 75 else 'High')

latest['performance_level'] = latest['avg_exam_score'].apply(assign_perf)

diff_map = {
    ('HIGH','Low'):'VERY_EASY', ('HIGH','Mid'):'EASY', ('HIGH','High'):'MEDIUM
    ('MEDIUM','Low'):'EASY', ('MEDIUM','Mid'):'MEDIUM', ('MEDIUM','High'):'MED
    ('LOW','Low'):'MEDIUM', ('LOW','Mid'):'MEDIUM_HARD', ('LOW','High'):'HARD'
```

```
}
blooms = {'VERY_EASY':'Remember','EASY':'Understand','MEDIUM':'Apply',
          'MEDIUM_HARD':'Analyze','HARD':'Evaluate/Create'}

latest['difficulty_level'] = latest.apply(lambda r: diff_map.get((r['risk_labe
latest['bloom_level'] = latest['difficulty_level'].map(blooms)

# Sample lecture content
SAMPLE_LECTURE_CONTENT = """
Introduction to Photosynthesis - Grade 8 Science
Photosynthesis is the process by which green plants use sunlight to synthesize
with carbon dioxide and water. The chemical equation: 6CO2 + 6H2O + light -> C
Key concepts: Light-dependent reactions in thylakoid membranes, Calvin Cycle i
chlorophyll absorbs red and blue light. Factors: light intensity, CO2, tempera
Plants use glucose for energy, growth, and starch storage. Photosynthesis prod
forms the base of food chains, and removes CO2 from atmosphere.
Experiment: Test for starch using iodine solution - blue-black indicates starc
"""

# Use extracted PDF content if available (from Part 1 / Section 3),
# otherwise fall back to the sample photosynthesis content
try:
    if full_content is not None and len(full_content.strip()) > 0:
        LECTURE_CONTENT = full_content
        print("[OK] Using extracted PDF lecture content for quiz generation.")
    else:
        LECTURE_CONTENT = SAMPLE_LECTURE_CONTENT
        print("[INFO] No PDF content found. Using sample lecture content (Phot
except NameError:
    LECTURE_CONTENT = SAMPLE_LECTURE_CONTENT
    print("[INFO] 'full_content' not defined. Using sample lecture content (Ph
    print("       Run Part 1 (Section 3: PDF Extraction) first to use your own


print("=" * 60)
print("SECTION 5: QUIZ GENERATION PIPELINE")
print("=" * 60)
```

```
[OK] Using extracted PDF lecture content for quiz generation.
============================================================
SECTION 5: QUIZ GENERATION PIPELINE
============================================================
```

In [33]:

```
# ============================================================
# QUIZ GENERATION FUNCTION
# ============================================================
def generate_quiz_gemini(lecture_content, grade, difficulty, bloom_level, n_qu
    """Generate quiz questions from lecture content using Gemini Flash API"""

    prompt = f"""You are an educational quiz generator for Sri Lankan school s

LECTURE CONTENT:
{lecture_content}
```

```
STUDENT CONTEXT:
- Grade: {grade}
- Difficulty Level: {difficulty}
- Bloom's Taxonomy Target: {bloom_level}

TASK:
Generate exactly {n_questions} multiple-choice questions STRICTLY based on the
Each question must:
- Be derived from the lecture material (not general knowledge)
- Match the {bloom_level} level of Bloom's taxonomy
- Be appropriate for Grade {grade} students in Sri Lanka
- Include 4 options (A, B, C, D) with exactly one correct answer
- Include a brief explanation referencing the lecture content

OUTPUT FORMAT: Return ONLY a valid JSON array. No markdown, no extra text.
[
  {{
    "question": "...",
    "option_a": "...",
    "option_b": "...",
    "option_c": "...",
    "option_d": "...",
    "correct_answer": "A/B/C/D",
    "explanation": "...",
    "bloom_level": "{bloom_level}"
  }}
]"""

    if not API_AVAILABLE:
        return None

    try:
        response = gemini_model.generate_content(prompt)
        text = response.text.strip()
        # Clean markdown code blocks if present
        text = re.sub(r'```json\s*', '', text)
        text = re.sub(r'```\s*', '', text)
        questions = json.loads(text)
        return questions
    except Exception as e:
        print(f"  [ERROR] API call failed: {e}")
        return None

# %%
# --- SAMPLE QUESTIONS (Fallback for demo mode) ---
SAMPLE_QUESTIONS = {
    'VERY_EASY': [
        {"question": "What is photosynthesis?", "option_a": "Process of making
        {"question": "True or False: Chlorophyll is green in color.", "option_
    ],
    'EASY': [
        {"question": "Explain why leaves appear green.", "option_a": "Chloroph
        {"question": "What are the raw materials needed for photosynthesis?",
```

```
    ],
    'MEDIUM': [
        {"question": "If a plant is placed in a dark room for 48 hours, what h
        {"question": "Using the equation from the lecture, how many molecules
    ],
    'MEDIUM_HARD': [
        {"question": "Compare and contrast the light-dependent and light-indep
    ],
    'HARD': [
        {"question": "Evaluate how increasing temperature beyond 40C would aff
    ],
}
```

In [34]:

```
# ============================================================
# GENERATE QUESTIONS FOR SAMPLE STUDENTS
# ============================================================
print("\n--- Generating Questions for Sample Students ---")

# Select 15 students: 5 per risk level
sample_students = []
for risk in ['LOW', 'MEDIUM', 'HIGH']:
    risk_students = latest[latest['risk_label'] == risk].head(2) #HEAD * 3 = S
    sample_students.append(risk_students)
sample_df = pd.concat(sample_students)

all_questions = []

for _, student in sample_df.iterrows():
    sid = student['student_id']
    grade = student['grade']
    difficulty = student['difficulty_level']
    bloom = student['bloom_level']

    print(f"\n  Student {sid} | Grade {grade} | Risk: {student['risk_label']}
          f"Difficulty: {difficulty} | Bloom: {bloom}")

    if API_AVAILABLE:
        questions = generate_quiz_gemini(LECTURE_CONTENT, grade, difficulty, b
        time.sleep(1)  # Rate limiting
    else:
        questions = SAMPLE_QUESTIONS.get(difficulty, SAMPLE_QUESTIONS['MEDIUM'

    if questions:
        for q in questions:
            q['student_id'] = sid
            q['grade'] = grade
            q['risk_label'] = student['risk_label']
            q['difficulty_level'] = difficulty
            all_questions.append(q)
        print(f"    -> Generated {len(questions)} questions")
    else:
        print(f"    -> [SKIP] No questions generated")
```

```python
questions_df = pd.DataFrame(all_questions)
print(f"\n[OK] Total questions generated: {len(questions_df)}")
questions_df.head()
```

```
--- Generating Questions for Sample Students ---

  Student S003 | Grade 7 | Risk: LOW | Difficulty: MEDIUM_HARD | Bloom: Analyze
    -> Generated 3 questions

  Student S004 | Grade 8 | Risk: LOW | Difficulty: MEDIUM_HARD | Bloom: Analyze
    -> Generated 3 questions

  Student S002 | Grade 10 | Risk: MEDIUM | Difficulty: EASY | Bloom: Understand
    -> Generated 3 questions

  Student S010 | Grade 9 | Risk: MEDIUM | Difficulty: EASY | Bloom: Understand
    -> Generated 3 questions

  Student S001 | Grade 10 | Risk: HIGH | Difficulty: EASY | Bloom: Understand
    -> Generated 3 questions

  Student S017 | Grade 6 | Risk: HIGH | Difficulty: EASY | Bloom: Understand
    -> Generated 3 questions

[OK] Total questions generated: 18
```

Out[34]:

	question	option_a	option_b	option_c	option_d	correct
0	A student wants to build a simple IoT device t...	Single-Board Computer (SBC) because it runs a ...	Microcontroller Board (MCU) because it is comp...	Apollo Guidance Computer because it has an inn...	A smartphone's System on a Chip (SoC) because ...	
1	Imagine a network of small weather sensors spr...	Wi-Fi, because it offers high bandwidth for se...	Ethernet, because it provides a reliable, wire...	LPWAN (Low-Power Wide-Area Network), such as L...	Cellular Communications (4G/5G), because it al...	
2	A Sri Lankan company wants to develop a new sm...	Access to well-documented resources and active...	The ability to offload meeting regulatory requ...	Better cost-effectiveness and scalability, all...	Reduced time to market due to ready-made softw...	
3	A school is developing a low-cost, battery-pow...	A Single-Board Computer (SBC) because it has h...	A Microcontroller board because it is more com...	A Single-Board Computer (SBC) because its abil...	A Microcontroller board because it offers mult...	
4	An environmental agency is deploying IoT devic...	Wi-Fi, because it offers high bandwidth for de...	Ethernet, as it provides reliable, high-speed ...	LoRaWAN, due to its long-range capabilities an...	Cellular (e.g., 5G), because it provides very ...	

Section 6: Quality Analysis of Generated Questions

In [35]:

```
# ============================================================
# QUALITY ANALYSIS
# ============================================================
print("\n" + "=" * 60)
print("SECTION 6: QUALITY ANALYSIS")
print("=" * 60)

# --- Content Fidelity Check ---
# Check if question keywords appear in the lecture content
lecture_words = set(LECTURE_CONTENT.lower().split())

def content_fidelity(question_text):
    q_words = set(question_text.lower().split())
    overlap = q_words.intersection(lecture_words)
    return len(overlap) / max(len(q_words), 1)

if len(questions_df) > 0:
    questions_df['content_fidelity'] = questions_df['question'].apply(content_
    questions_df['word_count'] = questions_df['question'].apply(lambda x: len(
```

```
    print(f"\nContent Fidelity Score: {questions_df['content_fidelity'].mean()
    print(f"Avg Question Length: {questions_df['word_count'].mean():.1f} words

# %%
# --- Quality Charts ---
if len(questions_df) > 0:
    fig, axes = plt.subplots(2, 3, figsize=(18, 10))

    # Chart 1: Bloom's level distribution
    bloom_counts = questions_df['bloom_level'].value_counts()
    axes[0, 0].pie(bloom_counts, labels=bloom_counts.index, autopct='%1.1f%%',
                   startangle=90, colors=plt.cm.Set3(np.linspace(0, 1, len(blo
    axes[0, 0].set_title("Bloom's Level Distribution", fontweight='bold')

    # Chart 2: Word count by difficulty
    diff_order = ['VERY_EASY', 'EASY', 'MEDIUM', 'MEDIUM_HARD', 'HARD']
    existing_diffs = [d for d in diff_order if d in questions_df['difficulty_l
    data_wc = [questions_df[questions_df['difficulty_level'] == d]['word_count
    if data_wc:
        bp = axes[0, 1].boxplot(data_wc, labels=existing_diffs, patch_artist=T
        colors_bp = ['#2ecc71', '#27ae60', '#f39c12', '#e67e22', '#e74c3c']
        for patch, c in zip(bp['boxes'], colors_bp[:len(bp['boxes'])]):
            patch.set_facecolor(c); patch.set_alpha(0.7)
    axes[0, 1].set_title('Word Count by Difficulty', fontweight='bold')
    axes[0, 1].tick_params(axis='x', rotation=45)

    # Chart 3: Content fidelity by difficulty
    fidelity_by_diff = questions_df.groupby('difficulty_level')['content_fidel
    fidelity_by_diff = fidelity_by_diff.reindex([d for d in diff_order if d in
    axes[0, 2].bar(fidelity_by_diff.index, fidelity_by_diff.values,
                   color='#3498db', edgecolor='white')
    axes[0, 2].set_title('Content Fidelity by Difficulty', fontweight='bold')
    axes[0, 2].set_ylabel('Fidelity Score')
    axes[0, 2].tick_params(axis='x', rotation=45)

    # Chart 4: Questions per risk category
    risk_q = questions_df.groupby(['risk_label', 'difficulty_level']).size().u
    risk_q.plot(kind='bar', ax=axes[1, 0], edgecolor='white')
    axes[1, 0].set_title('Questions per Risk Category', fontweight='bold')
    axes[1, 0].tick_params(axis='x', rotation=0)
    axes[1, 0].legend(fontsize=7)

    # Chart 5: Questions per grade
    grade_q = questions_df['grade'].value_counts().sort_index()
    axes[1, 1].bar(grade_q.index, grade_q.values, color='#9b59b6', edgecolor='
    axes[1, 1].set_title('Questions per Grade Level', fontweight='bold')
    axes[1, 1].set_xlabel('Grade')

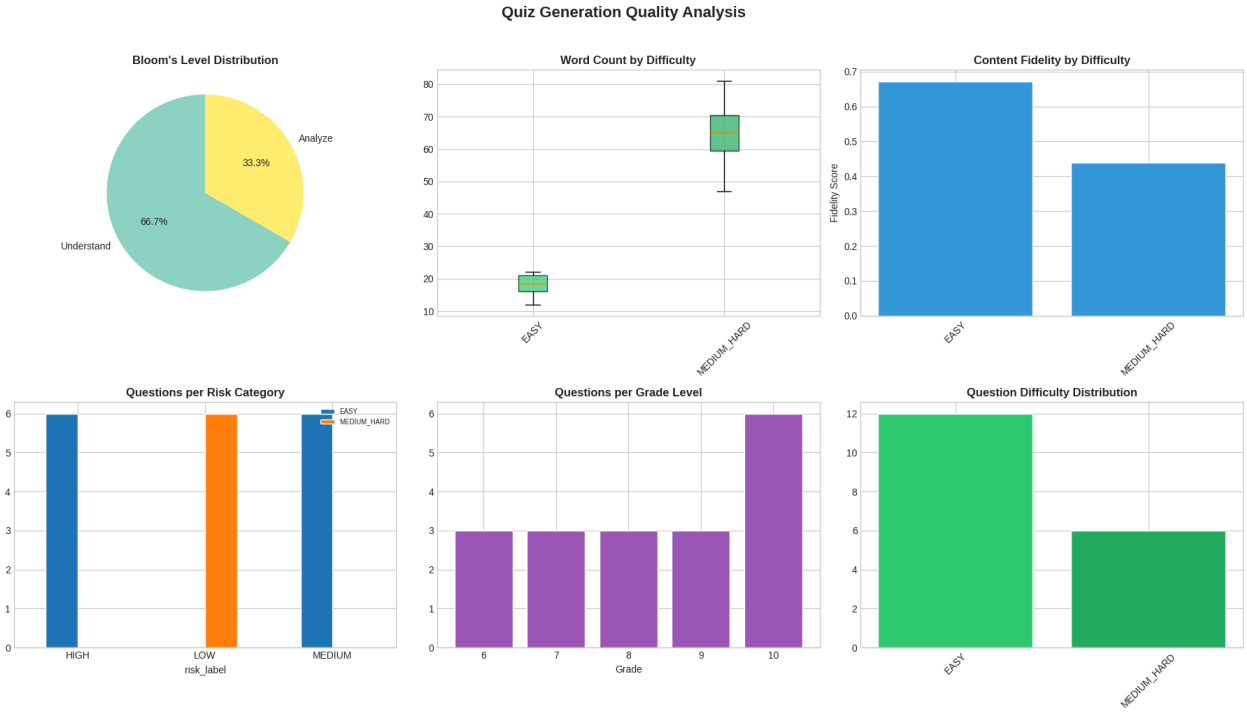
    # Chart 6: Difficulty distribution
    diff_q = questions_df['difficulty_level'].value_counts()
    diff_q = diff_q.reindex([d for d in diff_order if d in diff_q.index])
    axes[1, 2].bar(diff_q.index, diff_q.values,
```

```
                    color=['#2ecc71','#27ae60','#f39c12','#e67e22','#e74c3c'][:
                    edgecolor='white')
    axes[1, 2].set_title('Question Difficulty Distribution', fontweight='bold'
    axes[1, 2].tick_params(axis='x', rotation=45)

    fig.suptitle('Quiz Generation Quality Analysis', fontsize=16, fontweight='
    plt.tight_layout()
    plt.savefig('c2_chart3_quality_analysis.png', dpi=150, bbox_inches='tight'
    plt.show()
```

```
============================================================
SECTION 6: QUALITY ANALYSIS
============================================================

Content Fidelity Score: 0.594 (1.0 = perfect overlap)
Avg Question Length: 33.5 words
```



Section 7: Adaptive Difficulty Simulation

In [36]:

```
# ============================================================
# SECTION 7: SIMULATE ADAPTIVE RESPONSE LOOP (3 ROUNDS)
# ============================================================
print("\n" + "=" * 60)
print("SECTION 7: ADAPTIVE DIFFICULTY SIMULATION")
print("=" * 60)

difficulty_levels = ['VERY_EASY', 'EASY', 'MEDIUM', 'MEDIUM_HARD', 'HARD']

# Probability of correct answer based on risk and difficulty
# Higher risk students perform better on easier questions
correct_probs = {
    'LOW':    {'VERY_EASY': 0.95, 'EASY': 0.90, 'MEDIUM': 0.80, 'MEDIUM_HARD':
```

```
    'MEDIUM': {'VERY_EASY': 0.90, 'EASY': 0.80, 'MEDIUM': 0.65, 'MEDIUM_HARD':
    'HIGH':   {'VERY_EASY': 0.85, 'EASY': 0.70, 'MEDIUM': 0.50, 'MEDIUM_HARD':
}

# Simulate 3 rounds for each sample student
np.random.seed(42)
simulation_results = []

for _, student in sample_df.iterrows():
    sid = student['student_id']
    risk = student['risk_label']
    current_diff = student['difficulty_level'] if 'difficulty_level' in studer
    diff_idx = difficulty_levels.index(current_diff) if current_diff in diffic

    for round_num in range(1, 4):
        current_diff = difficulty_levels[diff_idx]
        prob = correct_probs[risk][current_diff]

        # Simulate 5 question answers
        n_correct = np.random.binomial(5, prob)
        accuracy = n_correct / 5

        simulation_results.append({
            'student_id': sid, 'risk_label': risk, 'round': round_num,
            'difficulty': current_diff, 'accuracy': accuracy, 'n_correct': n_c
        })

        # Adapt difficulty for next round
        if accuracy < 0.5:
            diff_idx = max(0, diff_idx - 1)  # Easier
        elif accuracy > 0.8:
            diff_idx = min(4, diff_idx + 1)  # Harder

sim_df = pd.DataFrame(simulation_results)

print(f"Simulation: {len(sample_df)} students x 3 rounds = {len(sim_df)} recor
print(f"\nAverage accuracy by round:")
print(sim_df.groupby('round')['accuracy'].mean().round(3))

# %%
# --- Simulation Charts ---
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Chart 1: Accuracy by difficulty level
acc_by_diff = sim_df.groupby('difficulty')['accuracy'].mean()
acc_by_diff = acc_by_diff.reindex([d for d in difficulty_levels if d in acc_by
axes[0, 0].bar(acc_by_diff.index, acc_by_diff.values,
               color=['#2ecc71','#27ae60','#f39c12','#e67e22','#e74c3c'][:len(
               edgecolor='white')
axes[0, 0].set_title('Average Accuracy by Difficulty Level', fontweight='bold'
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].tick_params(axis='x', rotation=45)
axes[0, 0].axhline(0.5, color='red', linestyle='--', alpha=0.5, label='50% thr
```

```
axes[0, 0].legend()

# Chart 2: Accuracy improvement across rounds
for risk in ['LOW', 'MEDIUM', 'HIGH']:
    risk_data = sim_df[sim_df['risk_label'] == risk]
    acc_by_round = risk_data.groupby('round')['accuracy'].mean()
    color = {'LOW': '#2ecc71', 'MEDIUM': '#f39c12', 'HIGH': '#e74c3c'}[risk]
    axes[0, 1].plot(acc_by_round.index, acc_by_round.values, 'o-',
                    linewidth=2, markersize=8, color=color, label=f'{risk} Ris
axes[0, 1].set_title('Accuracy Improvement Across Rounds', fontweight='bold')
axes[0, 1].set_xlabel('Round')
axes[0, 1].set_ylabel('Average Accuracy')
axes[0, 1].set_xticks([1, 2, 3])
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

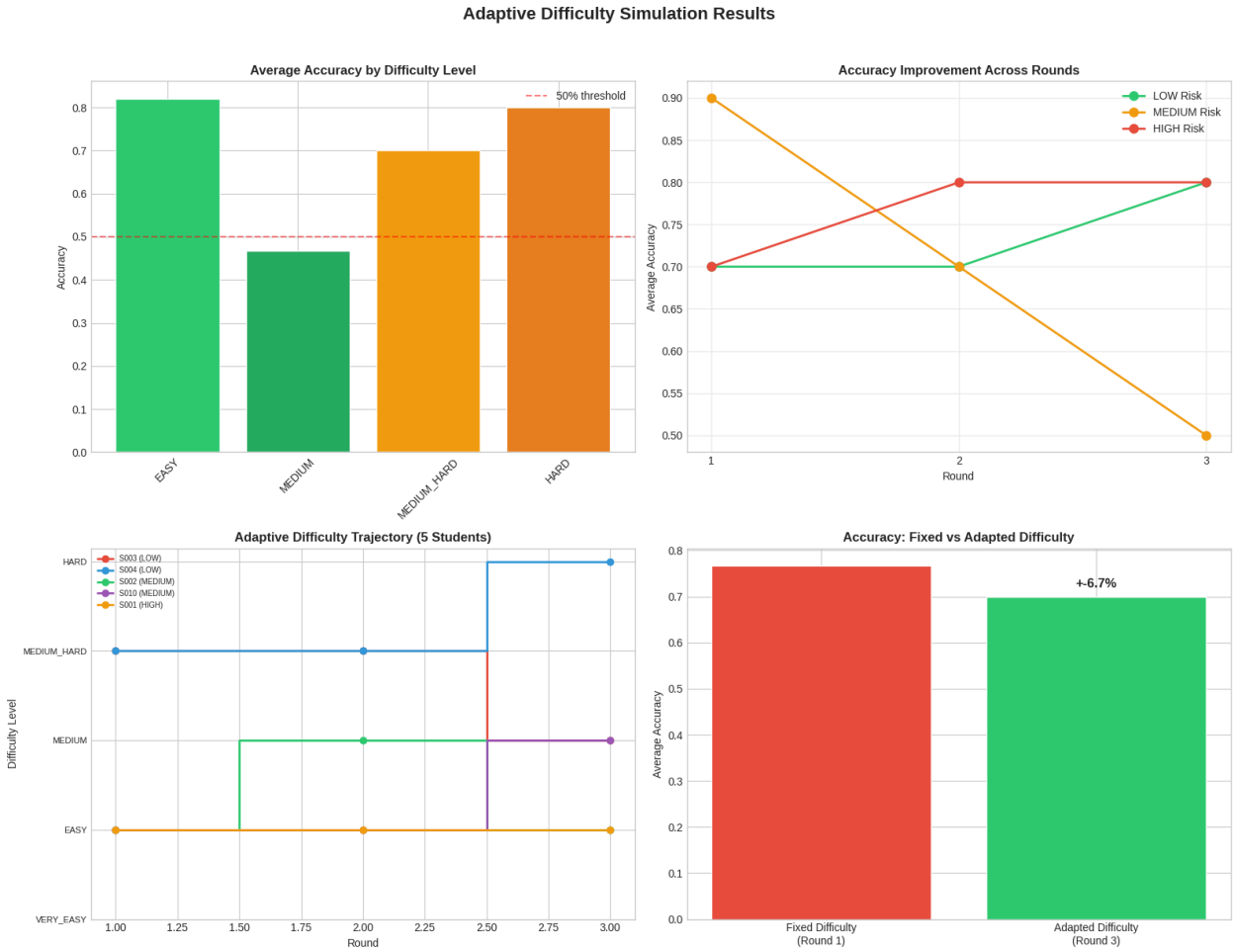
# Chart 3: Difficulty trajectory (5 sample students)
sample_sids = sim_df['student_id'].unique()[:5]
colors_line = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6', '#f39c12']
for i, sid in enumerate(sample_sids):
    s_data = sim_df[sim_df['student_id'] == sid]
    diff_nums = [difficulty_levels.index(d) for d in s_data['difficulty']]
    axes[1, 0].step(s_data['round'], diff_nums, where='mid', linewidth=2,
                    marker='o', color=colors_line[i], label=f'{sid} ({s_data["
axes[1, 0].set_yticks(range(5))
axes[1, 0].set_yticklabels(difficulty_levels, fontsize=8)
axes[1, 0].set_title('Adaptive Difficulty Trajectory (5 Students)', fontweight
axes[1, 0].set_xlabel('Round')
axes[1, 0].set_ylabel('Difficulty Level')
axes[1, 0].legend(fontsize=7)

# Chart 4: Stress reduction estimate
# Compare average accuracy with adapted vs fixed difficulty
fixed_acc = sim_df[sim_df['round'] == 1]['accuracy'].mean()
adapted_acc = sim_df[sim_df['round'] == 3]['accuracy'].mean()
axes[1, 1].bar(['Fixed Difficulty\n(Round 1)', 'Adapted Difficulty\n(Round 3)'
               [fixed_acc, adapted_acc], color=['#e74c3c', '#2ecc71'], edgecol
axes[1, 1].set_title('Accuracy: Fixed vs Adapted Difficulty', fontweight='bold
axes[1, 1].set_ylabel('Average Accuracy')
improvement = (adapted_acc - fixed_acc) * 100
axes[1, 1].text(1, adapted_acc + 0.02, f'+{improvement:.1f}%', ha='center', fo

fig.suptitle('Adaptive Difficulty Simulation Results', fontsize=16, fontweight
plt.tight_layout()
plt.savefig('c2_chart4_adaptive_simulation.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
============================================================
SECTION 7: ADAPTIVE DIFFICULTY SIMULATION
============================================================
Simulation: 6 students x 3 rounds = 18 records

Average accuracy by round:
round
1    0.767
2    0.733
3    0.700
Name: accuracy, dtype: float64
```



Section 8: End-to-End Pipeline Demo & Export

In [37]:

```
# ============================================================
# END-TO-END PIPELINE DEMO
# ============================================================
print("\n" + "=" * 60)
print("END-TO-END PIPELINE DEMO")
print("=" * 60)

demo_student = latest.iloc[0]
print(f"\n1. Student: {demo_student['student_id']}")
print(f"2. Grade: {demo_student['grade']}")
```

```
print(f"3. Wellbeing Score: {demo_student['composite_wellbeing_score']}")
print(f"4. Risk Label: {demo_student['risk_label']}")
print(f"5. Exam Score: {demo_student['avg_exam_score']} -> Performance: {demo_
print(f"6. Assigned Difficulty: {demo_student['difficulty_level']}")
print(f"7. Bloom's Target: {demo_student['bloom_level']}")
print(f"\n8. Generated Quiz:")

demo_qs = questions_df[questions_df['student_id'] == demo_student['student_id'
if len(demo_qs) > 0:
    for i, (_, q) in enumerate(demo_qs.iterrows(), 1):
        print(f"\n   Q{i}. {q['question']}")
        print(f"       A) {q['option_a']}")
        print(f"       B) {q['option_b']}")
        print(f"       C) {q['option_c']}")
        print(f"       D) {q['option_d']}")
        print(f"       Correct: {q['correct_answer']}")
        print(f"       Explanation: {q['explanation']}")
else:
    print("   [No questions available for this student]")

# %%
# --- Export ---
if len(questions_df) > 0:
    questions_df.to_csv('generated_quiz_questions.csv', index=False)
    print(f"\n[OK] Questions exported: generated_quiz_questions.csv ({len(ques

sim_df.to_csv('adaptive_simulation_results.csv', index=False)
print(f"[OK] Simulation exported: adaptive_simulation_results.csv ({len(sim_df

print("\n" + "=" * 60)
print("COLAB 2 COMPLETE: Adaptive Question Difficulty System built!")
print("=" * 60)
print("\nAll charts saved:")
print("  - c2_chart1_pdf_analysis.png")
print("  - c2_chart2_difficulty_mapping.png")
print("  - c2_chart3_quality_analysis.png")
print("  - c2_chart4_adaptive_simulation.png")
```

```
============================================================
END-TO-END PIPELINE DEMO
============================================================

1. Student: S001
2. Grade: 10
3. Wellbeing Score: 67.4
4. Risk Label: HIGH
5. Exam Score: 61 -> Performance: Mid
6. Assigned Difficulty: EASY
7. Bloom's Target: Understand

8. Generated Quiz:

   Q1. Which of the following best describes a Microcontroller (MCU) board in t
he context of IoT components?
       A) A complete computer built on a single board that runs a full operatin
g system like Linux.
       B) An integrated circuit containing a processor, memory, and I/O, design
ed for small, specific embedded applications and typically runs firmware.
       C) A specialized device solely for displaying information and receiving
complex user commands.
       D) A system primarily used for managing cloud infrastructure and perform
ing large-scale data analytics.
       Correct: B
       Explanation: According to slide 7, a Microcontroller (MCU) is 'An Integr
ated Circuit (chip) with a processor, memory and I/O (pins) built in... Intende
d for small, specific embedded system applications.' Slide 8 further states tha
t it 'Runs on firmware' and has 'No Operating System'.

   Q2. What is the main function of an actuator in an IoT system?
       A) To collect data from the environment, such as temperature or motion.
       B) To process complex calculations and run an operating system.
       C) To perform an action in the physical world, like moving a motor or tu
rning on a light.
       D) To establish wireless communication between different IoT devices.
       Correct: C
       Explanation: Slide 19 defines actuators as 'the outputs for the device—t
he motors, lights, and so on, which let your device do something to the outside
world.'

   Q3. Based on the lecture, what is a key difference between TCP (Transmission
Control Protocol) and UDP (User Datagram Protocol) regarding IoT communication?
       A) TCP is connectionless and lightweight, while UDP is connection-orient
ed and reliable.
       B) TCP requires a three-way handshake for setup and is reliable, whereas
UDP is connectionless and unreliable.
       C) UDP is exclusively used for sending data to the cloud, while TCP is o
nly for local device communication.
       D) Both TCP and UDP offer the same level of reliability and always use a
handshake for every transmission.
       Correct: B
       Explanation: Slide 24 explains the differences: 'TCP: Connection-oriente
d, reliable, requires three-way handshake for setup.' and 'UDP: Connectionless,
```

```
lightweight, unreliable, no handshake, lower overhead.'

[OK] Questions exported: generated_quiz_questions.csv (18 questions)
[OK] Simulation exported: adaptive_simulation_results.csv (18 records)

============================================================
COLAB 2 COMPLETE: Adaptive Question Difficulty System built!
============================================================

All charts saved:
  - c2_chart1_pdf_analysis.png
  - c2_chart2_difficulty_mapping.png
  - c2_chart3_quality_analysis.png
  - c2_chart4_adaptive_simulation.png
```